

Cahier des Charges

Projet Pluridisciplinaire (PPD)
M1 MIAGE - Université Paris Cité

Système RAG pour une Gestion du Cloud Efficiente

Master MIAGE - M1

12 juin 2026

Table des matières

1	Présentation du Projet	3
1.1	Contexte	3
1.2	Problématique	3
1.3	Objectif	3
2	Périmètre du Projet	3
2.1	Fonctionnalités incluses	3
2.2	Fonctionnalités exclues	4
3	Expression des Besoins	4
3.1	Acteurs	4
3.2	Cas d'utilisation	4
3.3	Recherche d'information	4
3.4	Recommandation	4
3.5	Analyse Green IT	4
4	Exigences Fonctionnelles	4
5	Exigences Non Fonctionnelles	5
5.1	Performance	5
5.2	Maintenabilité	5
5.3	Portabilité	5
5.4	Sobriété Numérique	5
6	Architecture Technique	5
6.1	Vue d'ensemble	5
6.2	Technologies retenues	5
6.3	Langages	5

6.4	LLM	5
6.5	Base vectorielle	6
6.6	Solutions comparées	6
7	Gestion des Données	6
7.1	Sources Documentaires	6
7.2	Cycle de traitement	6
8	Planning Prévisionnel	6
9	Critères de Validation	6

1 Présentation du Projet

1.1 Contexte

La transformation numérique conduit les entreprises à adopter massivement des solutions Cloud pour héberger leurs applications et leurs données.

Le choix d’une architecture Cloud adaptée est devenu complexe en raison de la diversité des solutions existantes :

- Cloud Public
- Cloud Privé
- Cloud Hybride
- Multi-Cloud
- Edge Computing

Ces choix doivent également prendre en compte plusieurs contraintes :

- Performance
- Coût
- Sécurité
- Data Residency
- Sobriété Numérique
- Green IT

Le projet vise à développer un assistant intelligent capable d’aider à la prise de décision concernant les architectures Cloud.

1.2 Problématique

Comment aider un décideur ou un architecte SI à choisir une architecture Cloud adaptée à ses besoins tout en respectant les contraintes de localisation des données et de sobriété numérique ?

1.3 Objectif

Concevoir et développer un système RAG (Retrieval-Augmented Generation) capable de :

- Rechercher des informations pertinentes dans une base documentaire.
- Fournir des réponses contextualisées.
- Recommander des architectures Cloud adaptées.
- Intégrer les notions de Green IT et de Data Residency.

2 Périmètre du Projet

2.1 Fonctionnalités incluses

Le système devra permettre :

- L’importation de documents techniques.
- Le découpage automatique des documents (chunking).
- La génération d’embeddings.
- Le stockage des embeddings dans une base vectorielle.
- La recherche sémantique.

- La génération de réponses via un LLM.
- L'aide à la décision sur les architectures Cloud.

2.2 Fonctionnalités exclues

- Déploiement en production.
- Gestion multi-utilisateurs avancée.
- Authentification complexe.

3 Expression des Besoins

3.1 Acteurs

- Architecte SI
- Consultant Cloud
- Responsable Infrastructure
- Étudiant ou chercheur

3.2 Cas d'utilisation

3.3 Recherche d'information

L'utilisateur soumet une question concernant une architecture Cloud.

3.4 Recommandation

Le système propose une recommandation basée sur la documentation analysée.

3.5 Analyse Green IT

Le système identifie les recommandations permettant de réduire l'impact environnemental.

4 Exigences Fonctionnelles

Réf.	Description
F01	Importer des documents PDF et texte.
F02	Découper les documents en chunks.
F03	Générer les embeddings.
F04	Stocker les embeddings dans une base vectorielle.
F05	Effectuer une recherche sémantique.
F06	Générer une réponse à partir du contexte récupéré.
F07	Fournir des recommandations Cloud.
F08	Intégrer les critères Green IT.
F09	Fonctionner localement autant que possible.

5 Exigences Non Fonctionnelles

5.1 Performance

- Temps de réponse inférieur à 5 secondes.
- Recherche efficace dans plusieurs milliers de documents.

5.2 Maintenabilité

- Code documenté.
- Architecture modulaire.

5.3 Portabilité

- Exécution sous Linux et Windows.
- Utilisation de Docker recommandée.

5.4 Sobriété Numérique

- Limiter les appels externes.
- Privilégier une exécution locale.
- Réduire les traitements inutiles.

6 Architecture Technique

6.1 Vue d'ensemble

Le système repose sur une architecture RAG composée des éléments suivants :

1. Documents techniques.
2. Module de Chunking.
3. Modèle d'Embeddings.
4. Base Vectorielle.
5. Moteur de Recherche.
6. LLM.
7. Interface Utilisateur.

6.2 Technologies retenues

6.3 Langages

- Python
- Mojo

6.4 LLM

Llama 3 exécuté localement via Ollama.

6.5 Base vectorielle

Qdrant (solution principale).

6.6 Solutions comparées

- Qdrant
- Pgvector
- FAISS

7 Gestion des Données

7.1 Sources Documentaires

Le référentiel documentaire principal est constitué du document :

- AWS Well-Architected Framework - Sustainability Pillar

D'autres documents techniques pourront être ajoutés afin d'enrichir la base de connaissances.

7.2 Cycle de traitement

1. Collecte des documents.
2. Découpage en chunks.
3. Génération des embeddings.
4. Indexation dans Qdrant.
5. Recherche sémantique.
6. Génération de réponse.

8 Planning Prévisionnel

Phase	Description
Étude	Analyse du RAG et des technologies.
Conception	Architecture et ADR.
Développement	Implémentation du pipeline RAG.
Tests	Validation fonctionnelle.
Comparaison	Qdrant, FAISS et Pgvector.
Documentation	Rédaction des livrables.

9 Critères de Validation

Le projet sera validé si :

- Le système RAG est opérationnel.
- Les réponses générées sont pertinentes.
- Les choix techniques sont justifiés.
- Les ADR sont rédigés.

- Les comparaisons sont réalisées.
- Les objectifs Green IT sont pris en compte.